- HW3
  - Question 1
  - Question 2
  - Question 3
  - Question 4
  - Question 5
  - Question 6
  - Question 7


# HW3

## Question 1

> When trying to run the program null, a segmentation fault occured

```
[[yuro@csci-gnode-03 HW3]$ ./null
 Segmentation fault (core dumped)
```

## Question 2

> After recompiling with "-g" flag for more info and running with gdp, gdp shows a segmentation fault occured

```
[yuro@csci-gnode-03 HW3]$ gdb null_2
GNU gdb (GDB) Red Hat Enterprise Linux 12.1-5.el9
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from null_2...
(gdb) run
Starting program: /csegrid/home/yuro/OS/OS_HW/HW3/null_2
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x00000000004011c9 in q1 () at functions.cpp:17
17              std::cout << *x_pointer << std::endl;   // print the dereferenced pointer
Missing separate debuginfos, use: dnf debuginfo-install glibc-2.34-231.el9_7.2.x86_64 libgcc-11.5.0-11.el9.x86_64 libstdc++-11.5.0-11.el9.x86_64
(gdb)
```

## Question 3

> After running the program with valgrind, it gives a report of what occured when running the program, as well as heap, leak, and error summaries.
>
> This report says that in invalid read was attempted and where. It also says how the process was terminated (due to a segmentation fault signal).

```
[[yuro@csci-gnode-03 HW3]$ valgrind --leak-check=yes ./null_2
==2322831== Memcheck, a memory error detector
==2322831== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2322831== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==2322831== Command: ./null_2
==2322831==
==2322831== Invalid read of size 4
==2322831==    at 0x4011C9: q1() (functions.cpp:17)
==2322831==    by 0x40119E: main (main.cpp:8)
==2322831==  Address 0x0 is not stack'd, malloc'd or (recently) free'd
==2322831==
==2322831==
==2322831== Process terminating with default action of signal 11 (SIGSEGV): dumping core
==2322831==  Access not within mapped region at address 0x0
==2322831==    at 0x4011C9: q1() (functions.cpp:17)
==2322831==    by 0x40119E: main (main.cpp:8)
==2322831==  If you believe this happened as a result of a stack
==2322831==  overflow in your program's main thread (unlikely but
==2322831==  possible), you can try to increase the size of the
==2322831==  main thread stack using the --main-stacksize= flag.
==2322831==  The main thread stack size used in this run was 8388608.
==2322831==
==2322831== HEAP SUMMARY:
==2322831==     in use at exit: 72,704 bytes in 1 blocks
==2322831==   total heap usage: 1 allocs, 0 frees, 72,704 bytes allocated
==2322831==
==2322831== LEAK SUMMARY:
==2322831==    definitely lost: 0 bytes in 0 blocks
==2322831==    indirectly lost: 0 bytes in 0 blocks
==2322831==      possibly lost: 0 bytes in 0 blocks
==2322831==    still reachable: 72,704 bytes in 1 blocks
==2322831==         suppressed: 0 bytes in 0 blocks
==2322831== Reachable blocks (those to which a pointer was found) are not shown.
==2322831== To see them, rerun with: --leak-check=full --show-leak-kinds=all
==2322831==
==2322831== For lists of detected and suppressed errors, rerun with: -s
==2322831== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
Segmentation fault (core dumped)
```

# Question 4

> When running the program with gdb, no errors are shown.

```
[yuro@csci-gnode-03 HW3]$ gdb question_4
GNU gdb (GDB) Red Hat Enterprise Linux 12.1-5.el9
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from question_4...
(gdb) run
Starting program: /csegrid/home/yuro/OS/OS_HW/HW3/question_4
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".
[Inferior 1 (process 2323930) exited normally]
Missing separate debuginfos, use: dnf debuginfo-install glibc-2.34-231.el9_7.2.x86_64 libgcc-11.5.0-11.el9.x86_64 libstdc++-11.5.0-11.el9.x86_64
```

> However when running with valgrind, it picks up that the heap memory wasn't freed before exiting.

```
[[yuro@csci-gnode-03 HW3]$ valgrind --leak-check=yes ./question_4
==2324022== Memcheck, a memory error detector
==2324022== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2324022== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==2324022== Command: ./question_4
==2324022==
==2324022==
==2324022== HEAP SUMMARY:
==2324022==     in use at exit: 4 bytes in 1 blocks
==2324022==   total heap usage: 2 allocs, 1 frees, 72,708 bytes allocated
==2324022==
==2324022== 4 bytes in 1 blocks are definitely lost in loss record 1 of 1
==2324022==    at 0x484682F: malloc (vg_replace_malloc.c:446)
==2324022==    by 0x40121D: q4() (functions.cpp:58)
==2324022==    by 0x4011AE: main (main.cpp:8)
==2324022==
==2324022== LEAK SUMMARY:
==2324022==    definitely lost: 4 bytes in 1 blocks
==2324022==    indirectly lost: 0 bytes in 0 blocks
==2324022==      possibly lost: 0 bytes in 0 blocks
==2324022==    still reachable: 0 bytes in 0 blocks
==2324022==         suppressed: 0 bytes in 0 blocks
==2324022==
==2324022== For lists of detected and suppressed errors, rerun with: -s
==2324022== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

# Question 5

> When the program for question 5 is run, the program exits normally.

```
[[yuro@csci-gnode-03 HW3]$ ./question_5
 [yuro@csci-gnode-03 HW3]$
```

> However, when run with valgrind, it notifies of an invalid write of size 4 due to no bytes after the block size of 400 that was allocated. i.e. an out of bounds write.

```
[[yuro@csci-gnode-03 HW3]$ valgrind --leak-check=yes ./question_5
==2324496== Memcheck, a memory error detector
==2324496== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2324496== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==2324496== Command: ./question_5
==2324496==
==2324496== Invalid write of size 4
==2324496==    at 0x401245: q5() (functions.cpp:72)
==2324496==    by 0x4011AE: main (main.cpp:8)
==2324496==  Address 0x4da5e10 is 0 bytes after a block of size 400 alloc'd
==2324496==    at 0x484682F: malloc (vg_replace_malloc.c:446)
==2324496==    by 0x401236: q5() (functions.cpp:71)
==2324496==    by 0x4011AE: main (main.cpp:8)
==2324496==
==2324496==
==2324496== HEAP SUMMARY:
==2324496==     in use at exit: 400 bytes in 1 blocks
==2324496==   total heap usage: 2 allocs, 1 frees, 73,104 bytes allocated
==2324496==
==2324496== 400 bytes in 1 blocks are definitely lost in loss record 1 of 1
==2324496==    at 0x484682F: malloc (vg_replace_malloc.c:446)
==2324496==    by 0x401236: q5() (functions.cpp:71)
==2324496==    by 0x4011AE: main (main.cpp:8)
==2324496==
==2324496== LEAK SUMMARY:
==2324496==    definitely lost: 400 bytes in 1 blocks
==2324496==    indirectly lost: 0 bytes in 0 blocks
==2324496==      possibly lost: 0 bytes in 0 blocks
==2324496==    still reachable: 0 bytes in 0 blocks
==2324496==         suppressed: 0 bytes in 0 blocks
==2324496==
==2324496== For lists of detected and suppressed errors, rerun with: -s
==2324496== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 0 from 0)
```

# Question 6

> When the program is run, a garbage value is printed

```
[[yuro@csci-gnode-03 HW3]$ ./question_6
6840
[yuro@csci-gnode-03 HW3]$
```

> When run with valgrind, it notifies of an invalid read due to no bytes in a block that was free'd.

```
[yuro@csci-gnode-03 HW3]$ valgrind --leak-check=yes ./question_6
==2327039== Memcheck, a memory error detector
==2327039== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2327039== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==2327039== Command: ./question_6
==2327039==
==2327039== Invalid read of size 4
==2327039==    at 0x401284: q6() (functions.cpp:89)
==2327039==    by 0x4011BE: main (main.cpp:8)
==2327039==  Address 0x4da5c80 is 0 bytes inside a block of size 400 free'd
==2327039==    at 0x4849B4C: free (vg_replace_malloc.c:989)
==2327039==    by 0x40127F: q6() (functions.cpp:88)
==2327039==    by 0x4011BE: main (main.cpp:8)
==2327039==  Block was alloc'd at
==2327039==    at 0x484682F: malloc (vg_replace_malloc.c:446)
==2327039==    by 0x40126F: q6() (functions.cpp:87)
==2327039==    by 0x4011BE: main (main.cpp:8)
==2327039==
0
==2327039==
==2327039== HEAP SUMMARY:
==2327039==     in use at exit: 0 bytes in 0 blocks
==2327039==   total heap usage: 3 allocs, 3 frees, 74,128 bytes allocated
==2327039==
==2327039== All heap blocks were freed -- no leaks are possible
==2327039==
==2327039== For lists of detected and suppressed errors, rerun with: -s
==2327039== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

# Question 7

> For this case, no tool is needed, the compiler notifies that a pointer that wasn't the original pointer allocated tried to be freed.

```
[yuro@csci-gnode-03 HW3]$ g++ main.cpp functions.cpp -o question_7 -g
functions.cpp: In function 'void q7()':
functions.cpp:104:13: warning: 'void free(void*)' called on pointer 'data' with nonzero offset 196 [-Wfree-nonheap-object]
  104 |         free(&data[49]);                                // free value in the middle of the array
      |         ~~~~^~~~~~~~~~~
functions.cpp:103:34: note: returned from 'void* malloc(size_t)'
  103 |         int* data = (int*) malloc(100 * sizeof(int));   // allocate array
      |                            ~~~~~~~^~~~~~~~~~~~~~~~~~~
```